CERTIK

Security Assessment

Zebec - Rust

CertiK Assessed on Mar 30th, 2024



CertiK Assessed on Mar 30th, 2024

Zebec - Rust

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES	ECOSYSTEM	METHODS
Platform	Solana (SOL)	Manual Review, Static Analysis

LANGUAGE	TIMELINE	KEY COMPONENTS
Rust	Delivered on 03/30/2024	N/A

CODEBASE

Zebec Github Repo

View All in Codebase Page

COMMITS

Pre: 938b8d6e58860f210ac1110ce6850399329e68ba

Final: 21784d6d434746ad3b48202d4e40570f3c6dc521

View All in Codebase Page

Highlighted Centralization Risks

- Contract upgradeability
- Privileged role can mint tokens

Vulnerability Summary



8	6	0	0	2	0
Total Findings	Resolved	Mitigated	Partially Resolved	Acknowledged	Declined

0	Critical		Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.
1	Major	1 Acknowledged	Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.
1	Medium	1 Resolved	Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.
1	Minor	1 Resolved	Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.
5	Informational	4 Resolved, 1 Acknowledged	Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | ZEBEC - RUST

Summary

Executive Summary

Vulnerability Summary

Codebase

Audit Scope

Approach & Methods

Review Notes

Out-of-Scope Components

External Dependencies

Privileged Functions

Findings

SRC-02 : Centralization Related Risks and Upgradability

MIG-01 : Missing `ZBC` and `ZBCN` Token Decimal Conversion

INS-01 : Missing Mint Authority Validation

INI-02 : Assignee and Assigning Time for `zbcn_mint` Mint Authority Role

LIB-01 : Potential Front-Running Risk with Initialize Instructions

MIG-02 : `ZBCN` Max Minted Amount

MIG-03 : Meaningless Variable Name

SRC-01 : `ZBC` and `ZBCN` Token Price

Optimizations

INI-01 : More `migrate_pda` Account Storage Space is Allocated

Appendix

Disclaimer

CODEBASE | ZEBEC - RUST

Repository

Zebec Github Repo

Commit

Pre: 938b8d6e58860f210ac1110ce6850399329e68ba

Final: 21784d6d434746ad3b48202d4e40570f3c6dc521

AUDIT SCOPE | ZEBEC - RUST

9 files audited ● 2 files with Acknowledged findings ● 1 file with Resolved findings ● 6 files without findings

ID	Repo	File	SHA256 Checksum
MIG	Zebec-protocol/migration-program	programs/migration-program/src/instructions/migrate_token.rs	9b184bf9a7087b0800edcf31a448fdfabec608117b95e956f5d3dec90065c609
LIB	Zebec-protocol/migration-program	programs/migration-program/src/lib.rs	78c15aed614a024e70a0442a6ac2af81c6e519e7485399e13face8bb86594f1e
INI	Zebec-protocol/migration-program	programs/migration-program/src/instructions/init_config.rs	9ab06f0fecfe6ada46db679376bf9fcc75903b8133fecc41c2141e1f6be7b795
EME	Zebec-protocol/migration-program	programs/migration-program/src/instructions/emergency_pause.rs	78c053bf7b360b6953957056bd953d2efb7a756d475ae1aa96bd62bca1adb56f
MOD	Zebec-protocol/migration-program	programs/migration-program/src/instructions/mod.rs	3944c8fe097b4e5c6eb936438251cb49be15797c56fe81973e48249514739e4c
UPD	Zebec-protocol/migration-program	programs/migration-program/src/instructions/update_admin.rs	84d14be5ba8a025e36ce41ee00e01ae83e08eab71b2d7037d009bbc6688c7f00
MIR	Zebec-protocol/migration-program	programs/migration-program/src/state/migrate.rs	6503a3a91c759abe9dc633201a678e1c8cbd7b4103af5ea719a0efadd6e95db5
MOS	Zebec-protocol/migration-program	programs/migration-program/src/state/mod.rs	b8b68a90b8bde0ab634c69dc79fc3231ca0c5787cc967cd6257ee3b571503faf
ERR	Zebec-protocol/migration-program	programs/migration-program/src/error.rs	57832792fdebe2134a33efdfc2018598545f455c109f7c1382fddc752d54767e

APPROACH & METHODS | ZEBEC - RUST

This report has been prepared for Zebec to discover issues and vulnerabilities in the source code of the Zebec - Rust project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Manual Review and Static Analysis techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

REVIEW NOTES | ZEBEC - RUST

Out-of-Scope Components

The token programs `ZBC` and `ZBCN`, utilized in the ongoing audit project, have been exempted from the current audit scope.

External Dependencies

The project mainly contains the following dependencies:

For `migration_program` program:

Dependency	Version
anchor-lang	0.28.0
anchor-spl	0.28.0
solana-security-txt	0.1.5

It should also be noted here that the code dependencies are in active development in the current auditing version and some of the keywords/functionality already deprecated in the newer version, for example, the `UncheckAccount` keyword. It is necessary to keep the dependencies up-to-date to avoid potential vulnerabilities.

The on-chain program can be upgradeable after the initial deployment due to Solana's features. Also, based on the unique rent mechanism in Solana, the balance in the account should be carefully set.

We assume these dependencies are valid and non-vulnerable factors and implement proper logic to collaborate with the current project.

Privileged Functions

In program `migration_program`, multiple roles are adopted to ensure the dynamic runtime updates of the project, which were specified in the finding **SRC-02**.

Any compromise to the privileged roles may allow the hacker to take advantage of this authority.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community. Any plan to invoke the aforementioned functions should also be considered to move to the execution queue of `Timelock` contract.

Also, the Solana platform allows for the possibility of upgrading its programs, with the default upgrade authority being the entity responsible for deployment. In situations where the program has upgradability features and the account of the upgrade authority becomes compromised, there is the potential for an unauthorized and malicious update to the program.

FINDINGS | ZEBEC - RUST



8	0	1	1	1	5
Total Findings	Critical	Major	Medium	Minor	Informational

This report has been prepared to discover issues and vulnerabilities for Zebec - Rust. Through this audit, we have uncovered 8 issues ranging from different severity levels. Utilizing the techniques of Manual Review & Static Analysis to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
SRC-02	**Centralization Related Risks And Upgradability**	**Centralization**	**Major**	**Acknowledged**
MIG-01	Missing `ZBC` And `ZBCN` Token Decimal Conversion	Logical Issue	Medium	Resolved
INS-01	Missing Mint Authority Validation	Logical Issue	Minor	Resolved
INI-02	Assignee And Assigning Time For `zbcn_mint` Mint Authority Role	Logical Issue	Informational	Resolved
LIB-01	Potential Front-Running Risk With Initialize Instructions	Volatile Code	Informational	Acknowledged
MIG-02	`ZBCN` Max Minted Amount	Logical Issue	Informational	Resolved
MIG-03	Meaningless Variable Name	Coding Style	Informational	Resolved
SRC-01	`ZBC` And `ZBCN` Token Price	Design Issue	Informational	Resolved

SRC-02 | CENTRALIZATION RELATED RISKS AND UPGRADABILITY

Category	Severity	Location	Status
Centralization	**Major**	**programs/migration-program/src/instructions/migrate_token.rs: 114~115; programs/migration-program/src/lib.rs: 25, 29**	**Acknowledged**

Description

In program `migration_program`, the role `admin` has privilege over the following instructions:

- `emergency_pause()`: maliciously pause the ZBC migration service.
- `update_admin()`: grant `admin` role to malicious account.

In program `migration_program`, the `ZBCN` role `mint::authority` has privilege over the following instructions:

- `mint_to()`: mint `ZBCN` to any account.

Any compromise to the privileged roles may allow the hacker to take advantage of this authority. The hackers may manipulate the cross-chain transfers. Such actions could result in financial losses for both the project and its users.

Also, the Solana platform allows for the possibility of upgrading its programs, with the default upgrade authority being the entity responsible for deployment. In situations where the program has upgradability features and the account of the upgrade authority becomes compromised, there is the potential for an unauthorized and malicious update to the program.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We recommend carefully managing the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multi-signature wallets.

Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (⅔, ⅗) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND

- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
  AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
  AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement;
  AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles;
  OR
- Remove the risky functionality.

Alleviation

[Zebec, 2024/03/29]: Issue acknowledged. The program will be made non-upgradable. An official announcement regarding the contract renouncement will be made through social media channels.

[CertiK, 2024/03/30]: It is suggested to implement the aforementioned methods to avoid centralized failure. Also, it strongly encourages the project team periodically revisit the private key security management of all addresses related to centralized roles.

MIG-01 | MISSING `ZBC` AND `ZBCN` TOKEN DECIMAL CONVERSION

Category	Severity	Location	Status
Logical Issue	Medium	programs/migration-program/src/instructions/migrate_token.rs: 97~98, 114~115	Resolved

Description

Based on the code logic below, it's clear that the value of the `amount` argument provided is used for both burning `ZBC` and minting `ZBCN`. Notably, there is no code logic for decimal conversion. Consequently, the minted amount of `ZBCN` does not match the expected value.

migrate_token.rs#L97

```
97      // Burn ZBC tokens.
98      burn(burn_ctx, amount)?;
```

migrate_token.rs#L114

```
114     // Mint equivalent ZBCN tokens that were burned.
115     mint_to(mint_ctx, amount)?;
```

Recommendation

Recommend refactoring the code to ensure accurate conversion of token decimals during the migration process.

Alleviation

[Zebec, 2024/03/29]: The Zebec team heeded the advise and resolved this issue at commit 21784d6d434746ad3b48202d4e40570f3c6dc521.

INS-01 | MISSING MINT AUTHORITY VALIDATION

Category	Severity	Location	Status
Logical Issue	Minor	programs/migration-program/src/instructions/init_config.rs: 19~28, 32; programs/migration-program/src/instructions/migrate_token.rs: 39~48	Resolved

Description

Based on the comment at L27, the `mint_authority` account has been assigned the role of mint authority for the `ZBCN` token. However, a current flaw in the code is the lack of a check to verify whether the `zbcn_mint` mint authority truly corresponds to the `mint_authority` PDA.

And after executing `init_config()` instruction, `migrate_pda.zbcn_mint` and `migrate_pda.mint_authority` cannot be updated.

init_config.rs#L19

```
19      #[account(
20          init,
21          payer = admin,
22          space = 8,
23          seeds = [b"zbcn_mint"],
24          bump
25      )]
26      /// CHECK: seeds has been checked
27      /// This account has mint authority for ZBCN tokens.
28      pub mint_authority: AccountInfo<'info>,
29      /// ZBC (new) mint account.
30      pub zbc_mint: Account<'info, Mint>,
31      /// ZBCN (old) mint account.
32      pub zbcn_mint: Account<'info, Mint>,
```

A similar issue also exists in `migrate_token.rs`.

migrate_token.rs#L39

```
39      /// ZBCN (new) mint account.
40      pub zbcn_mint: Account<'info, Mint>,
41      #[account(
42          mut,
43          seeds = [b"zbcn_mint"],
44          bump
45      )]
46      /// CHECK: seeds has been checked
47      /// This account has mint authority for ZBCN tokens.
48      pub mint_authority: AccountInfo<'info>,
```

Recommendation

Recommend refactoring the code to ensure that the `zbcn_mint` token mint authority account is `mint_authority` PDA.

Alleviation

[Zebec, 2024/03/29]: The Zebec team heeded the advise and resolved this issue at commit 21784d6d434746ad3b48202d4e40570f3c6dc521.

INI-02 ASSIGNEE AND ASSIGNING TIME FOR `zbcn_mint` MINT AUTHORITY ROLE

Category	Severity	Location	Status
Logical Issue	Informational	programs/migration-program/src/instructions/init_config.rs: 19~28, 32	Resolved

Description

According to the comment at L27, the `mint_authority` account has been designated the mint authority role for the `ZBCN` token. However, the current codebase lacks the necessary logic to assign the mint authority role for the `ZBCN` token to the `mint_authority` account.

```
19      #[account(
20          init,
21          payer = admin,
22          space = 8,
23          seeds = [b"zbcn_mint"],
24          bump
25      )]
26      /// CHECK: seeds has been checked
27      /// This account has mint authority for ZBCN tokens.
28      pub mint_authority: AccountInfo<'info>,
29      /// ZBC (new) mint account.
30      pub zbc_mint: Account<'info, Mint>,
31      /// ZBCN (old) mint account.
32      pub zbcn_mint: Account<'info, Mint>,
```

Recommendation

The audit team would like to ask Zebec team regarding the design logic behind the granting of the `ZBCN` mint authority role.

Alleviation

[Zebec, 2024/03/29]: The mint authority role is assigned to `mint_authority` pda during token creation.

LIB-01 POTENTIAL FRONT-RUNNING RISK WITH INITIALIZE INSTRUCTIONS

Category	Severity	Location	Status
Volatile Code	Informational	programs/migration-program/src/lib.rs: 17~19	Acknowledged

Description

The program `migration_program` can be initialized via `init_config` instruction and set up accounts with sensitive information for the corresponding program.

However, a malicious party can invoke `init_config` instructions, which will affect the program.

Recommendation

Recommend that the team review the process for deployment and initialization of programs. To make it more difficult for frontrunners to take advantage of the situation to gain ownership of contracts, the audit team recommends using extra gas during the initialization phase to make it costlier and less likely for front-running attacks to succeed(although the chance is low on Solana).

Depending on how the future implementations of the contract are structured, it would also be beneficial to restrict access to the initialization instructions to ensure that only intended users have access to the function.

Alleviation

[Zebec, 2024/03/29]: Issue acknowledged. Team review will be considered during both initialization and deployment phases.

MIG-02 | ZBCN MAX MINTED AMOUNT

Category	Severity	Location	Status
Logical Issue	Informational	programs/migration-program/src/instructions/migrate_token.rs: 84~88	Resolved

Description

According to the logic outlined in the following code, it is evident that only 10 billion `ZBCN` tokens are minted via the `migration-program`, implying that only 10 billion `ZBC` tokens undergo migration.

migrate_token.rs#L84

```
 84
// The migration program is one way ZBC -> ZBCN. The migration program will stop
once 10,000,000,000 ZBCN tokens are minted.

 85      require!(
 86          migrate_pda.total_migrated + amount <= 10_000_000_000 * 10u64.pow(
zbcn_decimals as u32),
 87          MigrationError::MaxMigrationReached
 88      );
```

Recommendation

The audit team would like to request confirmation from the Zebec team regarding whether the maximum supply of `ZBC` does not exceed 10 billion.

Alleviation

[Zebec, 2024/03/29]: The maximum supply for ZBC is capped at 10 billion. Each ZBC is burned during the migration process. After the migration process, the maximum supply for ZBCN will reach 100 billion, in accordance with the 1:10 migration ratio.

MIG-03 | MEANINGLESS VARIABLE NAME

Category	Severity	Location	Status
Coding Style	Informational	programs/migration-program/src/instructions/migrate_token.rs: 103, 111	Resolved

Description

The following `CPI` is utilized for minting `ZBCN`, however, the variable name `mint_zbc` fails to accurately convey the meaning of `MintTo` struct instance.

```
103     let mint_zbc = MintTo {
104         mint: zbcn_mint.to_account_info(),
105         to: sender_zbcn_ata.to_account_info(),
106         authority: mint_authority.to_account_info(),
107     };
108
109     let mint_ctx = CpiContext::new_with_signer(
110         token_program.to_account_info(),
111         mint_zbc,
112         mint_authority_seed,
113     );
```

Recommendation

Recommend using `mint_zbcn` to replce `mint_zbc`.

Alleviation

[Zebec, 2024/03/29]: The Zebec team heeded the advise and resolved this issue at commit 21784d6d434746ad3b48202d4e40570f3c6dc521.

SRC-01 | ZBC AND ZBCN TOKEN PRICE

Category	Severity	Location	Status
Design Issue	Informational	programs/migration-program/src/instructions/migrate_token.rs: 97 ~98, 114~115; programs/migration-program/src/lib.rs: 21	Resolved

Description

Based on the code logic below, it's evident that the provided `amount` argument value is used for both burning `ZBC` and minting `ZBCN`. This implies that the conversion rate between `ZBC` and `ZBCN` tokens is 1:1 when they have the same decimal precision.

migrate_token.rs#L97

```
97      // Burn ZBC tokens.
98      burn(burn_ctx, amount)?;
```

migrate_token.rs#L114

```
114     // Mint equivalent ZBCN tokens that were burned.
115     mint_to(mint_ctx, amount)?;
```

The holders of `ZBC` token assets accesses the `migrate_token()` function to convert a specified amount, rather than their entire holding, of `ZBC` to `ZBCN`. Therefore, the duration of the token migration is not predetermined. Throughout the migration process, the prices of `ZBC` and `ZBCN` may differ. In such instances, a 1:1 amount conversion could potentially reduce the value of the holder's assets if the price of `ZBCN` is lower than that of `ZBC`.

lib.rs#L21

```
21      pub fn migrate_token(ctx: Context<MigrateToken>, amount: u64) -> Result<()>
{
22          migrate_token::handler(ctx, amount)
23      }
```

Recommendation

The audit team would like to request confirmation from the Zebec team regarding whether the prices of both `ZBC` and `ZBCN` remain consistent throughout the entire token migration process.

Alleviation

[Zebec, 2024/03/29]: The migration ratio from ZBC to ZBCN has been adjusted to 1:10, with the decimal precision for ZBCN now set at 6. Pool creation for the token will occur only after all existing ZBC supply has been migrated to ensure price stability.

[CertiK, 2024/03/30]: The above mentioned migration solution means that users cannot trade `ZBCN` until all `ZBC` assets are migrated. The audit team recommend carefully handle this because the duration of token migration is actually controlled by `ZBC` holders.

OPTIMIZATIONS | ZEBEC - RUST

ID	Title	Category	Severity	Status
INI-01	More `migrate_pda` Account Storage Space Is Allocated	Coding Style	Optimization	Resolved

INI-01 MORE `migrate_pda` ACCOUNT STORAGE SPACE IS ALLOCATED

Category	Severity	Location	Status
Coding Style	Optimization	programs/migration-program/src/instructions/init_config.rs: 13	Resolved

Description

The space constraint at L13 serves to restrict the on-chain storage size of `migrate_pad`. In general, the `#[derive(InitSpace)]` macro is used to record `Migrate` storage size through adding an `INIT_SPACE` constant. This constant's value equals the sum of the sizes of the struct's fields with fixed byte sizes.

However, the usage of the `std::mem::size_of::<Migrate>()` function will get the size for the `Migrate` struct, including padding, exceeds the value of the `INIT_SPACE` constant. Consequently, the on-chain storage size of `migrate_pad` becomes larger than necessary, and will pay more rent lamports than actually needed.

init_config.rs#L10

```
10      #[account(
11          init,
12          payer = admin,
13          space = 8 + std::mem::size_of::<Migrate>(),
14          seeds = [b"migrate"],
15          bump
16      )]
17      /// PDA that stores the migration state and authority.
18      pub migrate_pda: Box<Account<'info, Migrate>>,
```

Proof of Concept

```rust
use anchor_lang::prelude::*;

#[derive(InitSpace)]
pub struct Migrate {
    /// admin Pubkey
    pub admin: Pubkey, // 32

    /// Flag indicating emergency pause status.
    pub emergency_pause: bool, // 1

    /// ZBC(old) mint key
    pub zbc_mint: Pubkey, // 32

    /// ZBCN mint key
    pub zbcn_mint: Pubkey, // 32

    /// Number of transactions processed by the program.
    pub transaction_count: u64, // 8

    /// Total number of tokens migrated.
    pub total_migrated: u64, // 8
}

#[test]
fn test_migrate_type_size() {
    assert!(std::mem::size_of::<Migrate>() > Migrate::INIT_SPACE, "");
    println!("Strut `Migrate` type size along with padding: {}", std::mem::size_of::
<Migrate>());
    println!("Struct `Migrate` INIT_SPACE value: {}", Migrate::INIT_SPACE);
}
```

Running on 64bits CPU, the output as below:

```
Strut `Migrate` type size along with padding: 120
Struct `Migrate` INIT_SPACE value: 113
```

Recommendation

Recommend refactoring the code to ensure accurate allocation of storage space and payment of fees.

Alleviation

[Zebec, 2024/03/29]: The Zebec team heeded the advise and resolved this issue at commit 21784d6d434746ad3b48202d4e40570f3c6dc521.

APPENDIX | ZEBEC - RUST

Finding Categories

Categories	Description
Coding Style	Coding Style findings may not affect code behavior, but indicate areas where coding practices can be improved to make the code more understandable and maintainable.
Volatile Code	Volatile Code findings refer to segments of code that behave unexpectedly on certain edge cases and may result in vulnerabilities.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

CertiK | Securing the Web3 World

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

